

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO1: *Analyze DBMS architecture, different data models, schema types, and design ER/EER diagrams, relational schemas for case-based scenarios by identifying entities and relationships. (BL1, BL2)*

Activity Tool : Short Answer Text(High)
Assessment Tool Weightage: 30%

Dr

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	UPS Product Tracking Information System UPS prides itself on having up-to-date information on the processing and current location of each shipped item. To do this, UPS relies on a company-wide information system. Shipped items are the heart of the UPS product tracking information system. Shipped items can be characterized by item number (unique), weight, dimensions, insurance amount, destination, and final delivery date. Shipped items are received into the UPS system at a single retail center. Retail centers are characterized by their type, uniqueID, and address. Shipped items make their way to their destination via one or more standard UPS transportation events (i.e., flights, truck deliveries). These transportation events are characterized by a unique scheduleNumber, a type	BL1 – Remember	5

	(e.g, flight, truck), and a deliveryRoute. Create an Entity Relationship diagram that captures this information about the UPS system.		
2	Course Management System Consider a university database for the scheduling of classrooms for final exams. This database could be modeled as the single entity set exam, with attributes course-name, section-number, room-number, and time. Alternatively, one or more additional entity sets could be defined, along with relationship sets to replace some of the attributes of the exam entity set, as • course with attributes name, department, and c-number • section with attributes s-number and enrollment, and dependent as a weak entity set on course • room with attributes r-number, capacity, and building Show an E-R diagram illustrating the use of all three additional entity sets listed.	BL2 – Understand	5
3	University Registrar's Office Management System A university registrar's office maintains data about the following entities: (a) courses, including number, title, credits, syllabus, and prerequisites; (b) course offerings, including course number, year, semester, section number, instructor(s), timings, and classroom; (c) students, including student-id, name, and program; (d) instructors, including identification number, name, department, and title. (e) Further, the enrollment of students in courses and grades awarded to students in each course they are enrolled for must be appropriately modeled. Construct an E-R diagram for the registrar's office.	BL2 – Understand	5

4	Hospital Management System Construct an E-R diagram for a hospital with a set of patients and a set of medical doctors. Associate with each patient a log of the various tests and examinations conducted. Construct appropriate tables for the above ER Diagram : Patient(SS#, name, insurance) Physician (name, specialization) Test-log(SS#, test-name, date, time) Doctor-patient (physician-name, SS#) Patient-history(SS#, test-name, date)	BL2 – Understand	5
5	Company Project Management System Construct an ER Diagram for Company having following details : Company organized into DEPARTMENT. Each department has unique name and a particular employee who manages the department. Start date for the manager is recorded. Department may have several locations. □ A department controls a number of PROJECT. Projects have a unique name, number and a single location. □ Company's EMPLOYEE name, ssno, address, salary, sex and birth date are recorded. An employee is assigned to one department, but may work for several projects (not necessarily controlled by dept). Number of hours/week an employee works on each project is recorded; The immediate supervisor for the employee. □ Employee's DEPENDENT are tracked for health insurance purposes (dependentname, birthdate, relationship to employee).	BL2 – Understand	5
6	Explain schema and instance with examples. Distinguish between logical schema, physical schema, and view schema.	BL1 – Remember	5
7	Identify the entities, attributes, and relationships for a Bank Management System and construct its ER diagram.	BL2 – Understand	5

8	Explain the Extended ER (EER) model. Describe the concepts of generalization, specialization, and aggregation.	BL2 – Understand	5
9	Design an EER diagram for a Hospital Management System incorporating inheritance and weak entities.	BL2 – Understand	5
10	Explain the role of a DBA (Database Administrator) and the responsibilities associated with the position.	BL1 – Remember	5

Code of Conduct:

I Nitthyendren JS 192421180 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Nitthyendren js

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	30%	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Linkages	25%	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Organization	20%	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Integration of Literature	15%	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response

Clarity	10%	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand
---------	-----	---	----------------------------------	---------------------------------------	-------------------------

ANSWER:

1. UPS Product Tracking Information System

UPS relies on a company-wide database to keep track of every shipped item as it moves from origin to destination. The system is built around three core entities that together capture the full journey of a package — where it entered the network, what physical and logistical properties it has, and how it physically travels to its destination.

Entities and Attributes:

- **Shipped Item:** item_number (PK, unique), weight, dimensions, insurance_amount, destination, final_delivery_date
- **Retail Center:** uniqueID (PK), type, address
- **Transportation Event:** scheduleNumber (PK), type (flight/truck), deliveryRoute

Relationships:

- **Receives** (Retail Center → Shipped Item): one-to-many, since each item enters through exactly one retail center, but a center receives many items over time
- **Routed_Via** (Shipped Item ↔ Transportation Event): many-to-many, because a single item may need multiple transportation events (e.g., a truck to the airport, then a flight, then another truck) to complete its journey, and each transportation event carries many different items simultaneously

Since items typically pass through more than one transportation leg, this many-to-many relationship could optionally be enhanced with an attribute like

sequence_number to record the order in which a shipment passes through each event, which is important for accurately tracking a package's current location at any given time.

2. Course Management System

Rather than storing all exam-scheduling data in a single flat entity, this design normalizes the information into three connected entities, which reduces redundancy and better reflects real-world structure — a section belongs to a course, and an exam for that section takes place in a room at a specific time.

Entities and Attributes:

- **Course:** name, department, c-number (PK)
- **Section:** s-number, enrollment — a weak entity, since a section only has meaning in the context of its parent course

- **Room:** r-number (PK), capacity, building

Relationships:

- **Course–Section:** one-to-many, identifying relationship (a section cannot exist without belonging to a course)
- **Exam** (Section ↔ Room): a relationship set carrying the attribute **time**, representing when a given section's final exam is scheduled in a specific room

This design is more flexible than the single flat entity approach because it avoids duplicating course and room details every time a new section is added, and it makes it easy to query, for example, "which rooms are booked during a given time slot" without scanning redundant data.

3. University Registrar's Office Management System

The registrar's office needs to track the full academic lifecycle — courses that exist, specific offerings of those courses in a given term, students enrolled, and instructors teaching them — along with how students perform in each course they take.

Entities and Attributes:

- **Course:** course_number (PK), title, credits, syllabus, plus a self-referencing **prerequisite** relationship (a course can require one or more other courses)
- **Course_Offering:** year, semester, section_number, timings, classroom — a weak entity dependent on Course
- **Student:** student_id (PK), name, program
- **Instructor:** instructor_id (PK), name, department, title

Relationships:

- **Offers** (Course → Course_Offering): one-to-many, identifying
- **Teaches** (Instructor ↔ Course_Offering): many-to-many, since a course offering can have co-instructors and an instructor teaches multiple offerings
- **Enrolls** (Student ↔ Course_Offering): many-to-many, carrying the attribute **grade**, since a grade is only meaningful in the context of a specific student taking a specific offering

The **prerequisite** relationship deserves special mention — it's a recursive relationship on the Course entity itself, since a Course can reference other Courses as required prior coursework.

4. Hospital Management System

This system tracks the interactions between patients, physicians, and the diagnostic tests they undergo. Because patients typically see multiple doctors and undergo repeated tests over time, both core relationships are many-to-many.

Entities and Attributes:

- **Patient:** SS# (PK), name, insurance
- **Physician:** name, specialization
- **Test:** test-name, date, time

Relationships:

- **Doctor-Patient:** many-to-many, since a patient may be treated by several physicians and each physician treats many patients
- **Test-log:** many-to-many, since a patient undergoes many tests over time

Corresponding Tables:

Patient(SS#, name, insurance)

Physician(name, specialization)

Test-log(SS#, test-name, date, time)

Doctor-patient(physician-name, SS#)

Patient-history(SS#, test-name, date)

The **Patient-history** table effectively acts as a simplified log linking a patient to the tests they've had over time, and could be extended with a **result** attribute to record test outcomes, which is often useful in real hospital systems for tracking diagnosis timelines.

5. Company Project Management System

This is a classic ER modeling scenario that captures the structure of a company: departments overseeing employees and projects, employees working across projects, supervisory hierarchies, and dependents for insurance purposes.

Entities and Attributes:

- **Department:** dname (PK), dnumber, manager_start_date, plus a multivalued attribute for locations
- **Project:** pname (PK), pnumber, location
- **Employee:** ssno (PK), name, address, salary, sex, birthdate
- **Dependent:** dependent_name, birthdate, relationship — a weak entity dependent on Employee

Relationships:

- **Manages** (Department ↔ Employee): one-to-one — one employee manages one department
- **Controls** (Department → Project): one-to-many
- **Works_For** (Employee → Department): many-to-one — each employee belongs to exactly one department
- **Works_On** (Employee ↔ Project): many-to-many, carrying the attribute **hours_per_week**
- **Supervises** (Employee → Employee): a recursive one-to-many relationship representing the reporting hierarchy
- **Has_Dependent** (Employee → Dependent): one-to-many, identifying

Because department locations are multivalued (a department can span several offices), they're typically modeled either as a multivalued attribute directly on Department, or broken out into a separate **Dept_Locations** table if the database needs to query locations independently — the latter is generally preferred in practice for easier indexing and querying.

6. Schema vs Instance

A **schema** is the overall structural design of a database — essentially a blueprint that defines what tables exist, what columns they contain, and how they relate to one another. It is analogous to the type definition of a variable in programming and generally remains stable over time, changing only when the database is intentionally redesigned. An **instance**, by contrast, is the actual content of the database at any given moment — the specific rows of data currently stored — and it changes constantly as data is inserted, updated, or deleted through everyday operations.

Example: A schema might define Student(id, name, age) as a table structure. An instance would be the actual rows like (101, "Ravi", 20) and (102, "Priya", 21) currently sitting in that table.

Three Levels of Schema:

- **Physical schema:** describes how data is physically stored on disk — file organization, indexing methods, storage allocation. This is the lowest level and is typically managed by the DBMS and hardware layer.
- **Logical schema:** describes the structure of the data itself — tables, columns, data types, and relationships — independent of how it's physically stored. This is the level most database designers and administrators work with directly.
- **View schema** (external schema): describes how individual users or applications see a customized subset of the data, often hiding irrelevant columns or combining data from multiple tables. Different users can have different views over the same underlying logical schema.

This three-level separation, known as the **three-schema architecture**, is important because it provides **data independence** — changes at one level (e.g., how data is physically stored) don't require changes at another level (e.g., how applications query the data).

7. Bank Management System

A bank's database needs to track customers, the accounts and loans they hold, the branches that service them, and the staff who work at each branch.

Entities and Attributes:

- **Customer:** cust_id (PK), name, address, phone
- **Account:** acc_no (PK), type (savings/current), balance
- **Branch:** branch_id (PK), name, address
- **Loan:** loan_no (PK), amount
- **Employee:** emp_id (PK), name, role

Relationships:

- **Owns** (Customer ↔ Account): many-to-many, since joint accounts allow multiple customers to own one account, and a customer can hold multiple accounts
- **Maintains** (Branch → Account): one-to-many
- **Takes** (Customer ↔ Loan): many-to-many, since loans can also be co-signed by multiple customers
- **Grants** (Branch → Loan): one-to-many

- **Employs** (Branch → Employee): one-to-many

In a more complete design, this model could be extended with a **Transaction** entity (transaction_id, date, amount, type) linked to Account, to capture the actual deposit and withdrawal history rather than just the current balance — this is essential for any real banking system that needs an audit trail.

8. Extended ER (EER) Model

The Extended Entity-Relationship (EER) model adds a set of higher-level modeling constructs on top of the basic ER model, allowing designers to represent more complex, real-world relationships between entities that share commonalities or hierarchical structure.

- **Generalization:** A bottom-up process where two or more entities that share common attributes are combined into a single, more general higher-level entity. For example, if a company has separate entities for **Car** and **Truck** that share attributes like registration_number and manufacturer, they can be generalized into a single **Vehicle** entity.
- **Specialization:** The reverse, top-down process, where a higher-level entity is divided into two or more lower-level sub-entities based on distinguishing characteristics. For example, an **Employee** entity might be specialized into **Manager** (with an additional attribute like team_size) and **Engineer** (with an additional attribute like tech_stack).
- **Aggregation:** Treats a relationship set itself as a higher-level entity so that it can participate in further relationships with other entities. This is useful when a relationship needs its own relationship — for example, if a "Works_On" relationship between Employee and Project needs to be further connected to a **Manager** entity that oversees that specific work assignment, aggregation allows Works_On to be treated as an entity in itself.

Generalization and specialization can also be classified further based on **disjointness** (whether an entity instance can belong to only one sub-class or multiple) and **completeness** (whether every instance of the higher-level entity must belong to at least one sub-class), which are additional refinements often expected in a full EER discussion.

9. EER Diagram for Hospital Management System

This design demonstrates both **inheritance**, through a specialization hierarchy of people working at or visiting the hospital, and **weak entities**, for records that only make sense in the context of a specific patient.

Entities and Attributes:

- **Person** (superclass): id (PK), name, address
 - Specializes into **Patient**: adds insurance
 - Specializes into **Staff**: adds employee_id, salary
 - **Staff** further specializes into **Doctor**: adds specialization
 - **Staff** further specializes into **Nurse**: adds shift
- **Appointment**: date, time, reason — a weak entity dependent on Patient
- **Prescription**: medicine, dosage — a weak entity dependent on the Doctor-Patient treatment relationship

Relationships:

- **Treats** (Doctor ↔ Patient): many-to-many
- **Has_Appointment** (Patient → Appointment): one-to-many, identifying

This two-level specialization (Person → Staff → Doctor/Nurse) is a good example of **multilevel inheritance** in EER modeling, since Doctor and Nurse inherit attributes from both Staff and, transitively, from Person. It also illustrates **disjoint specialization**, since a given Staff member is either a Doctor or a Nurse, not both simultaneously.

10. Role of a Database Administrator (DBA)

A **Database Administrator (DBA)** is the individual or team responsible for the overall control, management, and upkeep of a database system, ensuring it remains secure, efficient, and available to the organization that depends on it. Because a database is often one of the most critical assets an organization owns, the DBA's role carries significant responsibility across both design-time and day-to-day operational tasks.

Key Responsibilities:

- **Schema definition**: Creating the original database schema by writing the necessary data definition language (DDL) statements
- **Storage structure and access method definition**: Deciding how data is physically stored and indexed for efficient retrieval
- **Schema and physical organization modification**: Updating the schema or storage structure over time as the organization's requirements evolve, without disrupting existing applications
- **Granting authorization for data access**: Managing user accounts, roles, and permissions to ensure only authorized users can view or modify specific data — a critical part of database security
- **Routine maintenance**: Performing regular backups, planning and executing recovery procedures in the event of failure, monitoring system performance, and tuning queries or indexes to maintain efficiency

Beyond these technical duties, a DBA is also often responsible for enforcing **data integrity constraints**, coordinating with application developers on schema changes, and ensuring compliance with data protection regulations — making the role both a technical and organizational responsibility within any enterprise that relies on a database system.